`std::iterator_interface`

Document #:	P2727R3
Date:	2022-11-20
Project:	Programming Language C++
Audience:	LEWG-I LEWG

Reply-to:	Zach Laine <whatwasthataddress@gmail.com>

Contents

- 1 Changelog
  - 1.1 Changes since R0
  - 1.2 Changes since R1
  - 1.3 Changes since R2
- 2 Motivation
  - 2.1 The story for everyday users writing STL iterators is not great
  - 2.2 Writing STL iterators is very useful
  - 2.3 Iterator adaptation
- 3 Proposed design
  - 3.1 Add `iterator_interface_access`
  - 3.2 Add `proxy_arrow_result`
  - 3.3 Add `iterator_interface` itself
  - 3.4 Add free operators
  - 3.5 Add `operator==` overload
  - 3.6 Add `proxy_iterator_interface`
  - 3.7 Add a feature test macro
  - 3.8 Incompatibility of input iterators with non-`ranges` algorithms
  - 3.9 Design notes
    - 3.9.1 Nested typedefs in the dependent-base case
    - 3.9.2 The name `iterator_interface`
- 4 Performance
- 5 References

§ 1 Changelog

§ 1.1 Changes since R0

- Added discussion about nested typedefs in dependent-base cases.
- Added an alternate name.
- Added discussion about performance.
- Typos.

1.2 Changes since R1

Changes recommended by the 2023-03-21 LEWG telecon review, and the subsequent reflector discussion:

— Change how (and whether) `iterator_category` is defined.

— Disable `operator->` when the user specifies `void` for `Pointer`, or when `Reference` is not a language reference.

— Add support for `operator<=>`.

— Return `void` from `operator++(int)` for input iterators.

1.3 Changes since R2

— Radically changed the implementation. Instead of using CRTP, the template deduces `this`, and the formerly-hidden friends are no longer hidden.

2 Motivation

2.1 The story for everyday users writing STL iterators is not great

Writing STL iterators is surprisingly hard. There are a lot of things that can subtly go wrong. It is also very tedious, which of course makes it error-prone.

Iterators have numerous typedefs and operations, even though all the operations of a given iterator can be implemented in terms of at most four operations (and usually only three). Writing all the other operations yields very similar-looking code that is hard to review, and all but requires that you write full-coverage tests for each iterator.

As an example, say you wanted an iterator that allowed you to iterate over repetitions of a sequence of characters, like:

```
repeated_chars_iterator first("foo", 3, 0); // 3 is the length of "foo", 0 is this iterator's position.
repeated_chars_iterator last("foo", 3, 7);  // Same as above, but now the iterator's position is 7.
std::string result;
std::copy(first, last, std::back_inserter(result));
assert(result == "foofoof");
```

Here's how you might implement it, with and without this proposal:

Before

```
struct repeated_chars_iterator
{
    using value_type = char;
    using difference_type =
        std::ptrdiff_t;
    using pointer = char const *;
    using reference = char const;
    using iterator_category =
        std::random_access_iterator_tag;

    constexpr repeated_chars_iterator()
        :
        first_(nullptr),
        size_(0),
        n_(0)
    {}
    constexpr repeated_chars_iterator(
        char const * first,
        difference_type size,
        difference_type n) :
        first_(first),
        size_(size),
        n_(n)
    {}

    constexpr reference operator*()
        const
    {
        return first_[n_ % size_];
```

After

```
struct repeated_chars_iterator :
    std::iterator_interface<std::random_access_iterator_tag,
        char, char>
{
    constexpr repeated_chars_iterator() :
        first_(nullptr),
        size_(0),
        n_(0)
    {}
    constexpr repeated_chars_iterator(char const * first,
                                      difference_type size,
                                      difference_type n) :
        first_(first),
        size_(size),
        n_(n)
    {}

    constexpr char operator*() const
    {
        return first_[n_ % size_];
    }
    constexpr repeated_chars_iterator &
    operator+=(std::ptrdiff_t i)
    {
        n_ += i;
        return *this;
    }
    constexpr auto operator-(
        repeated_chars_iterator other) const
```

Before

```
    }

    constexpr value_type operator[](
        difference_type n) const
    {
        return first_[(n_ + n) %
        size_];
    }

    constexpr repeated_chars_iterator &
        operator++()
    {
        ++n_;
        return *this;
    }
    constexpr repeated_chars_iterator
    operator++(int)noexcept
    {
        repeated_chars_iterator retval =
        *this;
        ++*this;
        return retval;
    }
    constexpr repeated_chars_iterator &
    operator+=(difference_type n)
    {
        n_ += n;
        return *this;
    }

    constexpr repeated_chars_iterator &
        operator--()
    {
        --n_;
        return *this;
    }
    constexpr repeated_chars_iterator
    operator--(int)noexcept
    {
        repeated_chars_iterator retval =
        *this;
        --*this;
        return retval;
    }
    constexpr repeated_chars_iterator &
    operator-=(difference_type n)
    {
        n_ -= n;
        return *this;
    }

    friend constexpr bool operator==(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return lhs.first_ == rhs.first_
        &&
            lhs.n_ == rhs.n_;
    }
    friend constexpr bool operator!=(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return !(lhs == rhs);
    }
    friend constexpr bool operator<(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return lhs.first_ == rhs.first_
        &&
            lhs.n_ < rhs.n_;
```

After

```
        {
            return n_ - other.n_;
        }

    private:
        char const * first_;
        difference_type size_;
        difference_type n_;
    };
```

Before	After

```
    }
    friend constexpr bool operator<=(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return lhs == rhs || lhs < rhs;
    }
    friend constexpr bool operator>(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return rhs < lhs;
    }
    friend constexpr bool operator>=(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return lhs <= rhs;
    }

    friend constexpr
        repeated_chars_iterator
    operator+(repeated_chars_iterator
        lhs,
              difference_type rhs)
    {
        return lhs += rhs;
    }
    friend constexpr
        repeated_chars_iterator
    operator+(difference_type lhs,
              repeated_chars_iterator
        rhs)
    {
        return rhs += lhs;
    }
    friend constexpr
        repeated_chars_iterator
    operator-(repeated_chars_iterator
        lhs,
              difference_type rhs)
    {
        return lhs -= rhs;
    }
    friend constexpr difference_type
        operator-(
        repeated_chars_iterator lhs,
        repeated_chars_iterator rhs)
    {
        return lhs.n_ - rhs.n_;
    }

private:
    char const * first_;
    difference_type size_;
    difference_type n_;
};
```

I used that motivating example in the README file for the Boost library this proposal is based on. One day, someone pointed out that there was a bug in the “before” version of the iterator. The iterator was used extensively in yet another proposed Boost library of mine, and yet this bug was never actually exercised that I know of. This underscores an important point: with very large APIs like the std iterators, one is very likely to use lots of copy pasta, and one is very unlikely to fully test each element of the API.

I’ve reintroduced the bug here for funsies; see how long it takes you to find it by inspection.

§ 2.2 Writing STL iterators is very useful

Some examples of useful iterators you may want to write:

— Checking iterators that do certain checks only in debug builds.

— Iterators that adapt legacy container-like types for use with STL algorithms and containers.

— *Ad hoc*, single-application iterators.

Because writing iterators is so much work at the moment, most of us avoid it whenever possible. So *ad hoc* use cases for iterators almost always go unfulfilled. Here's an example of one such *ad hoc* iterator use case.

Say you have two or more sequences that you want to treat as a single sequence:

```
std::vector<int> a;
std::list<int> b;
```

You'd use something like the `views::concat` proposed in [P2542R2]:

```
for (auto && x : concat(a, b)) {
    // ...
}
```

That works well, as long as there exists a common reference type among the iterators used within the `views::concat` implementation. What if there isn't? What if the `value_types` of the various ranges you pass to `concat()` are essentially compatible, but nevertheless have no common reference type?

For example, what if I want to concatenate two ranges, one of which is a range of `int` and the other of which is a range of:

```
struct my_int
{
    my_int() = default;
    explicit my_int(int v) : value_{v} {}
    int value_;
};
```

Clearly, `std::common_reference_with<my_int, int>` is `false`. However, if I also had some simple way of projecting from either one to `int`, like this:

```
int as_int(int i) { return i; }
int as_int(my_int i) { return i.value_; }
```

I would still expect to be able to construct some view over the concatenation of `a` and `b` – even if it were read-only.

This example may seem contrived, but it is not. While implementing an algorithm to support Unicode operations on strings, I ran in to a case in which a certain step `S` of the algorithm required iterating over two containers `A a` and `B b`, where `A::iterator::reference` and `B::iterator::reference` were different enough that there was no common reference, but `A::value_type` and `B::value_type` were similar enough that I could just call a function (analogous to `as_int()` above) that could project both of them to a single type. To make matters worse, I could not even break `S` up into the `a`-part and the `b`-part, because `S` could not consider elements in isolation; it required examining adjacent elements in the concatenation of `a` and `b`.

None of the views proposed for the standard has ever supported projections. I don't consider this a defect; it's probably the wrong kind of customization for a general-purpose view. So, what to do? Write an iterator that fits this one-off, special case, of course! Using Boost.STLInterfaces, an implementation of everything in this proposal and more, I wrote something like this iterator. To keep all the Unicode-y bits out of our way, I'm showing the `int`/`my_int` analogous solution.

```
template<typename T, typename Proj, typename R1, typename R2>
struct concat_iter : boost::stl_interfaces::proxy_iterator_interface<
                         std::bidirectional_iterator_tag,
                         T>
{
    using first_iterator = decltype(std::declval<R1 &>().begin());
    using second_iterator = decltype(std::declval<R2 &>().begin());
    struct end_tag{};

    concat_iter() : in_r1_(false) {}
    concat_iter(first_iterator it,
                first_iterator r1_last,
                second_iterator r2_first,
                Proj proj) :
        r1_last_(r1_last),
        it1_(it),
```

```
        r2_first_(r2_first),
        it2_(r2_first),
        proj_(proj),
        in_r1_(true)
    {
        if (it1_ == r1_last_)
            in_r1_ = false;
    }
    concat_iter(second_iterator it,
                first_iterator r1_last,
                second_iterator r2_first,
                Proj proj,
                end_tag) :
        r1_last_(r1_last),
        it1_(r1_last),
        r2_first_(r2_first),
        it2_(it),
        proj_(proj),
        in_r1_(false)
    {}

    concat_iter & operator++() noexcept
    {
        if (in_r1_) {
            assert(it1_ != r1_last_);
            ++it1_;
            if (it1_ == r1_last_)
                in_r1_ = false;
        } else {
            ++it2_;
        }
        return *this;
    }

    concat_iter & operator--() noexcept
    {
        if (!in_r1_) {
            if (it2_ == r2_first_) {
                in_r1_ = true;
                --it1_;
            } else {
                --it2_;
            }
        } else {
            --it1_;
        }
        return *this;
    }

    T operator*() const noexcept
    {
        return in_r1_ ? proj_(*it1_) : proj_(*it2_);
    }

    friend bool operator==(concat_iter lhs, concat_iter rhs)
    {
        return lhs.in_r1_ == rhs.in_r1_ &&
               (lhs.in_r1_ ?
                lhs.it1_ == rhs.it1_ : lhs.it2_ == rhs.it2_);
    }

    using base_type =
        boost::stl_interfaces::proxy_iterator_interface<
            std::bidirectional_iterator_tag,
            T>;
    using base_type::operator++;
    using base_type::operator--;

private:
    first_iterator r1_last_;
    first_iterator it1_;
    second_iterator r2_first_;
```

```
    second_iterator it2_;
    Proj proj_;
    bool in_r1_;
};
```

That's a pleasantly low amount of code to write, and I was able to avoid implementing most of the boilerplate bits – the operations that need to exist for my type to be conforming bidirectional iterator, but I which I will not explicitly use.

In case you were wondering about the `T` template parameter, that just gets around the need for some potentially-messy metaprogramming to determine what `concat_iter`'s `value_type` is – it's whatever you specify for `T`.

With the iterator in hand, it becomes trivial to construct a `concat_view`, and a `concat()` function:

```
template<typename T, typename R1, typename R2>
struct concat_view
    : std::view_interface<concat_view<T, R1, R2>>
{
    using iterator = concat_iter<T, R1, R2>;

    concat_view(iterator first, iterator last) :
        first_(first), last_(last)
    {}

    iterator begin() const noexcept { return first_; }
    iterator end() const noexcept { return last_; }

private:
    iterator first_;
    iterator last_;
};

template<typename T, typename R1, typename R2, typename Proj>
auto concat(R1 & r1, R2 & r2, Proj proj)
{
    using iterator = concat_iter<T, Proj, R1, R2>;
    return concat_view<T, R1, R2>(
        iterator(r1.begin(), r1.end(), r2.begin(), proj),
        iterator(r2.end(), r1.end(), r2.begin(), proj,
                 iterator::end_tag{}));
}
```

Finally, here it is at the point of use:

```
std::vector<int> vec = { 0, 58, 48 };
std::list<my_int> list = { 80, 39 };
auto const v = concat<int>(vec, list, [](auto x) { return as_int(x); });
assert(std::ranges::find(v, 42) == v.end());
assert(std::ranges::find(v, 58) == std::next(v.begin()));
```

Some things to note here:

— The amount of boilerplate is very small.

— The total amount of code is very small.

— This abstraction allows me to concentrate on the algorithm I want to do on the concatenated sequence, rather than the details of dealing with boundary conditions between the underlying sequences, *in addition to* the algorithm itself. Imagine if I had to have logic like that found in `concat_iter::operator++()` *intermixed* with the algorithm code.

When opportunities like this come up – that is, opportunities to write custom iterators – the dominant consideration is what it costs to write an iterator. If the cost is too high, we don't write one, and the algorithm that we would have written in terms of the unwritten iterator becomes much more complicated (and thus error-prone). If the cost is low, we will start to notice opportunities to write custom iterators in many more places. One thing in particular that easy-to-write iterators enable is easy-to-write views and view adaptors.

§ 2.3 Iterator adaptation

Sometimes, you want to take an existing iterator and adapt it for a different use. Since you already have all the iterator operations defined on it, it would be useful to simply use those, and fill in the missing ones, or perhaps replace the ones that

you want to work differently.

For example, if we wanted to make a filtering iterator for something like `filter_view`, we could write it like this:

```
template<typename Pred>
struct filtered_int_iterator : boost::stl_interfaces::iterator_interface<
                                   std::forward_iterator_tag,
                                   int>
{
    filtered_int_iterator() : it_(nullptr) {}
    filtered_int_iterator(int * it, int * last, Pred pred) :
        it_(it),
        last_(last),
        pred_(std::move(pred))
    { it_ = std::find_if(it_, last_, pred_); }

    // A forward iterator based on iterator_interface usually requires
    // three user-defined operations.  since we are adapting an existing
    // iterator (an int *), we only need to define this one.  The others are
    // implemented by iterator_interface, using the underlying int *.
    filtered_int_iterator & operator++()
    {
        it_ = std::find_if(std::next(it_), last_, pred_);
        return *this;
    }

    // It is really common for iterator adaptors to have a base() member
    // function that returns the adapted iterator.
    int * base() const { return it_; }

private:
    // Provide access to base_reference.
    friend boost::stl_interfaces::access;

    // These functions are picked up by iterator_interface, and used to
    // implement any operations that you don't define above.  They're not
    // called base() so that they do not collide with the base() member above.
    constexpr decltype(auto) base_reference(this auto& self) noexcept
    { return self.it_; }

    int * it_;
    int * last_;
    Pred pred_;
};
```

Though this implementation is limited to `int*`, it's pretty easy to see that you could change it to take an `Iterator` template parameter instead, and after replacing `int*` with `Iterator` throughout, everything would "just work".

§ 3 Proposed design

The proposal is to add a base template that will ease writing iterators, in the same way that `std::view_interface` eases the writing of views today.

§ 3.1 Add `iterator_interface_access`

Before looking at the main `iterator_interface` template, we need to look at some of the bits that allow that template to function. First, `iterator_interface_access`. `iterator_interface_access` is used by the adaptation logic to get access to the underlying iterator being adapted, which is usually private. The user just needs to friend `iterator_interface_access` from the iterator they write (just like the friend declaration in the `filtered_int_iterator` example above).

```
struct iterator_interface_access
{
  template<typename D>
  static constexpr auto base(D& d) noexcept
    -> decltype(d.base_reference())
  {
    return d.base_reference();
  }
  template<typename D>
```

```
  static constexpr auto base(const D& d) noexcept
    -> decltype(d.base_reference())
  {
    return d.base_reference();
  }
};
```

3.2 Add `proxy_arrow_result`

Next, `proxy_arrow_result`, a specialization of which is used as the default `pointer` for proxy iterators created using the `proxy_iterator_interface` alias template (shown a bit later).

```
template<typename T>
  requires is_object_v<T>
struct proxy_arrow_result
{
  constexpr proxy_arrow_result(const T& value) noexcept(
    noexcept(T(value))) :
    value_(value)
  {}
  constexpr proxy_arrow_result(T&& value) noexcept(
    noexcept(T(std::move(value)))) :
    value_(std::move(value))
  {}

  constexpr const T* operator->() const noexcept { return &value_; }
  constexpr T* operator->() noexcept { return &value_; }

private:
  T value_;
};
```

3.3 Add `iterator_interface` itself

```
template<class D1, class D2 = D1>
  concept base-3way =                                  // exposition only
                requires    (D1    d1,    D2    d2)    {    iterator_interface_access::base(d1)    <=>
        iterator_interface_access::base(d2); };

template<class D1, class D2 = D1>
  concept iter-sub = requires (D1 d1, D2 d2) {      // exposition only
    typename D1::difference_type;
    {d1 - d2} -> convertible_to<typename D1::difference_type>;
  };

template<class Pointer, class Reference, class T>
  requires is_pointer_v<Pointer>
    decltype(auto) make-iterator-pointer(T&& value) // exposition only
      { return addressof(value); }

template<class Pointer, class Reference, class T>
  auto make-iterator-pointer(T&& value)              // exposition only
    { return Pointer(std::forward<T>(value)); }

template<
  class IteratorConcept,
  class ValueType,
  class Reference = ValueType&,
  class Pointer = ValueType*,
  class DifferenceType = ptrdiff_t>
class iterator_interface
{
public:
  using iterator_concept = IteratorConcept;
  using iterator_category = see below;
  using value_type = remove_const_t<ValueType>;
  using reference = Reference;
  using pointer = conditional_t<
    is_same_v<iterator_concept, output_iterator_tag>>,
    void,
    Pointer>;
```

```
  using difference_type = DifferenceType;

  constexpr decltype(auto) operator*(this auto&& self)
    requires requires { *iterator_interface_access::base(self); } {
    return *iterator_interface_access::base(self);
  }

  constexpr auto operator->(this auto&& self)
    requires (!same_as<pointer, void>) && is_reference_v<reference> && requires { *self; } {
      return make-iterator-pointer<pointer, reference>(*self);
    }

   constexpr decltype(auto) operator[](this auto const& self, difference_type n) requires requires {
        self + n; } {
    auto retval = self;
    retval = retval + n;
    return *retval;
  }

  constexpr decltype(auto) operator++(this auto& self)
        requires requires { ++iterator_interface_access::base(self); } && (!requires { self +=
        difference_type(1); }) {
      ++iterator_interface_access::base(self);
      return self;
    }
   constexpr decltype(auto) operator++(this auto& self) requires requires { self += difference_type(1);
        } {
    return self += difference_type(1);
  }
  constexpr auto operator++(this auto& self, int) requires requires { ++self; } {
    if constexpr (is_same_v<IteratorConcept, input_iterator_tag>){
      ++self;
    } else {
      auto retval = self;
      ++self;
      return retval;
    }
  }
  constexpr decltype(auto) operator+=(this auto& self, difference_type n)
    requires requires { iterator_interface_access::base(self) += n; } {
      iterator_interface_access::base(self) += n;
      return self;
    }

  constexpr decltype(auto) operator--(this auto& self)
        requires requires { --iterator_interface_access::base(self); } && (!requires { self +=
        difference_type(1); }) {
      --iterator_interface_access::base(self);
      return self;
    }
  constexpr decltype(auto) operator--(this auto& self) requires requires { self += -difference_type(1);
        } {
    return self += -difference_type(1);
  }
  constexpr auto operator--(this auto& self, int) requires requires { --self; } {
    auto retval = self;
    --self;
    return retval;
  }
  constexpr decltype(auto) operator-=(this auto& self, difference_type n) requires requires { self += -
        n; } {
    return self += -n;
  }
};
```

The nested type `iterator_category` is defined if and only if `IteratorConcept` is derived from `forward_iterator_tag`. In that case, `iterator_category` is defined as follows:

— If `is_reference_v<ReferenceType>` is `false`, `iterator_category` denotes `input_iterator_tag`.

— Otherwise, if `derived_from<IteratorConcept, random_access_iterator_tag>` is `true`, `iterator_category` denotes `random_access_iterator_tag`.

— Otherwise, if `derived_from<IteratorConcept, bidirectional_iterator_tag>` is `true`, `iterator_category` denotes `bidirectional_iterator_tag`.

— Otherwise, `iterator_category` denotes `forward_iterator_tag`.

Note that this follows the semantics of `zip_transform_view::iterator`; see https://eel.is/c++draft/range.zip.transform.iterator#1 .

3.4 Add free operators

Additionally, we want several free operators, shown here. In addition to the constraints shown, they require that `D`, `D1`, and `D2` are derived from specializations of `iterator_interface`.

```
template<class D>
  constexpr auto operator+(D it, difference_type n) requires requires { it += n; } {
    return it += n;
  }
template<class D>
  constexpr auto operator+(difference_type n, D it) requires requires { it += n; } {
    return it += n;
  }

template<class D1, class D2>
  constexpr auto operator-(D1 lhs, D2 rhs)
     requires requires { iterator_interface_access::base(lhs) - iterator_interface_access::base(rhs); }
         {
      return iterator_interface_access::base(lhs) - iterator_interface_access::base(rhs);
    }
template<class D>
  constexpr auto operator-(D it, typename D::difference_type n) requires requires { it += -n; } {
    return it += -n;
  }

template<class D1, class D2>
  constexpr auto operator<=>(D1 lhs, D2 rhs) requires base-3way<D1, D2> || iter-sub<D1, D2> {
    if constexpr (base-3way<D1, D2>) {
      return iterator_interface_access::base(lhs) <=> iterator_interface_access::base(rhs);
    } else {
      using diff_type = typename D1::difference_type;
      const diff_type diff = rhs - lhs;
      return diff < diff_type(0) ? strong_ordering::less :
        diff_type(0) < diff ? strong_ordering::greater :
        strong_ordering::equal;
    }
  }
template<class D1, class D2>
  constexpr bool operator<(D1 lhs, D2 rhs) requires iter-sub<D1, D2> {
    return (lhs - rhs) < typename D1::difference_type(0);
  }
template<class D1, class D2>
  constexpr bool operator<=(D1 lhs, D2 rhs) requires iter-sub<D1, D2> {
    return (lhs - rhs) <= typename D1::difference_type(0);
  }
template<class D1, class D2>
  constexpr bool operator>(D1 lhs, D2 rhs) requires iter-sub<D1, D2> {
    return (lhs - rhs) > typename D1::difference_type(0);
  }
template<class D1, class D2>
  constexpr bool operator>=(D1 lhs, D2 rhs) requires iter-sub<D1, D2> {
    return (lhs - rhs) >= typename D1::difference_type(0);
  }
```

3.5 Add `operator==` overload

Additionally, we want free a `operator==` (and compiler-provided `operator!=`), shown here. In addition to the constraints shown, they require that `D1` and `D2` are derived from specializations of `iterator_interface`. This is so the overload can pick up comparisons of iterators of different, but interoperable types (like `std::vector<int>::const_iterator` and `std::vector<int>::iterator`). The full constraint is that one iterator is convertible to the other (in either direction), and either: 1) their adapted bases are comparable; or 2) a `D2` is subtractable from a `D1`.

I know those constraints look oddly specific; here's why they are the way they are:

Every category of iterator besides output need to define `operator==`. For this reason, each user-defined iterator derived from a specialization of `iterator_interface` will define its own `operator==`, unless it is an output iterator, *or* unless it is a random access (or contiguous) iterator. The exception for random access/contiguous is because `operator-`, which the user must define for a random access/contiguous iterator, can be used to implement `operator==` – for two iterators i1 and i2, just check if `i1 - i2 == 0`. This explains part of the constraints; this free overload is the implementation for `operator==` for random access/contiguous iterators.

For the base-equality-comparable part, similar logic applies: we don't want the user to have to define `operator==` when the underlying adapted iterator undoubtedly already has it (unless it's an output iterator, of course).

```
template<class D1, class D2>
  concept base-iter-comparable =              // exposition only
    requires (D1 d1, D2 d2) {
      iterator_interface_access::base(d1) == iterator_interface_access::base(d2);
    };

template<class D1, class D2>
  constexpr bool operator==(D1 lhs, D2 rhs)
    requires (is_convertible_v<D2, D1> || is_convertible_v<D1, D2>) &&
             (base-iter-comparable<D1, D2> || iter-sub<D1>) {
      if constexpr (base-iter-comparable<D1, D2>) {
        return (iterator_interface_access::base(lhs) ==
                iterator_interface_access::base(rhs));
      } else if constexpr (iter-sub<D1>) {
        return (lhs - rhs) == typename D1::difference_type(0);
      }
    }
```

§ 3.6 Add `proxy_iterator_interface`

Finally, there's an alias that makes it easier to define proxy iterators:

```
template<
    class IteratorConcept,
    class ValueType,
    class Reference = ValueType,
    class DifferenceType = ptrdiff_t>
using proxy_iterator_interface = iterator_interface<
    IteratorConcept,
    ValueType,
    Reference,
    proxy_arrow_result<Reference>,
    DifferenceType>;
```

§ 3.7 Add a feature test macro

Add the feature test macro `__cpp_lib_iterator_interface`.

§ 3.8 Incompatibility of input iterators with non-`ranges` algorithms

Based on LEWG telecon and reflector feedback, I changed the way `iterator_interface` works in the input iterator case. In particular, `operator++(int)` now returns `void`, and there is no `iterator_category` nested type.

This breaks use of non-`ranges` algorithms. Given input iterators `it` and `end` of some type `I` made using `iterator_interface`, `std::copy(it, end, /*...*/)` is ill-formed, though `std::ranges::copy(it, end, /*...*/)` works fine.

I consider this to be a design defect. I was careful to make `iterator_interface` in such a way that it works with `ranges` and pre-`ranges` algorithms alike, and this is no longer true for the input iterator case. There are likely more pre-`ranges` algorithms out there than `ranges` ones, including a significant number that do not appear in the standard.

The design presented here adheres very closely to the `std::input_iterator` concept in the input iterator case. That is, it tries not to provide any API outside of that concept – that's why `operator++(int)` returns `void` and why it does not define `iterator_category`.

The `void`-return for `operator++(int)` exists because an input iterator is not required to be copyable, according to the `std::input_iterator` concept. For a given iterator type `I` made using `iterator_interface`, if `I` is copyable, we could detect that and make `operator++(int)` do the same thing it does for all the other iterator categories, and return `I`. It would still model the `std::input_iterator` concept.

It would also still model the `std::input_iterator` concept if we defined `iterator_category` unconditionally.

This requires discussion and a poll.

3.9 Design notes

Since we're using a base template, any operation that is provided by default that you do not want, say because it has the wrong semantics, or because you know of a more efficient way to do it, you can simply provided that operation in your type, and the operation you would have inherited from the `iterator_interface` gets hidden.

To get interoperability between user-created `const_iterator` and `iterator` types, the user must make `iterators` convertible to `const_iterators`. You can't automate everything.

Here is a handy table listing all the user-provided operations necessary to implement all the various iterator concepts. This is every user-defined operation, even though no one iterator requires all of them. A later table will indicate which operations are needed when implementing which iterator concept. In the table, `Iter` is a user-defined type derived from `iterator_interface`; `i` and `i2` are objects of type `Iter`; `reference` is the type passed as the `Reference` template parameter to `iterator_interface`; `pointer` is the type passed as the `Pointer` template parameter to `iterator_interface`; and `n` is a value of type `difference_type`.

Expression	Return Type	Semantics	Notes
`*i`	Convertible to `reference`.	Dereferences `i` and returns the result.	*Precondition:* `i` is dereferenceable.
`i == i2`	Contextually convertible to `bool`.	Returns true if and only if `i` and `i2` refer to the same value.	*Precondition:* `(i, i2)` is in the domain of `==`.
`i2 - i`	Convertible to `difference_type`.	Returns `n`.	*Precondition:* there exists a value `n` of type `difference_type` such that `i + n == i2`.
`++i`	`Iter &`	Increments `i`.	
`--i`	`Iter &`	Decrements `i`.	

`i += n`	`Iter &`	`difference_type m = n;`
		`if (m >= 0)`
		`  while (m--) ++i;`
		`else`
		`  while (m++) --i;`	

This table shows which user-definable operations must be defined to meet the requirements of each of the iterator concepts. `i`, `i2`, and `n` have the same meaning here as they do in the previous table.

Concept	Operations

`input_iterator`	`*i`
	`i == i2`
	`++i`

`output_iterator`	`*i` `++i`

`forward_iterator`	`*i`
	`i == i2`
	`++i`

`bidirectional_iterator`	`*i`
	`i == i2`
	`++i`
	`--i`

`random_access_iterator/contiguous_iterator`	`*i` `i - i2`

Concept	Operations
	`i += n`

3.9.1 Nested typedefs in the dependent-base case

During the 2023-01-17 Library Evolution telecon, a question was raised about whether the nested typedefs provided by `iterator_interface` (`iterator_category`, etc.) would be defined in the derived iterator type, if the `iterator_interface` base class depended on a template parameter. It turns out they do, as far as I can tell. I already had at least one such dependent-base-class type in my unit tests for Boost.STLInterfaces, and I added another. Here is the code:

```
template<typename ValueType>
struct basic_random_access_iter_dependent
    : boost::stl_interfaces::iterator_interface<
          std::random_access_iterator_tag,
          ValueType>
{
    basic_random_access_iter_dependent() {}
    basic_random_access_iter_dependent(ValueType * it) : it_(it) {}

    ValueType & operator*() const { return *it_; }
    basic_random_access_iter_dependent & operator+=(std::ptrdiff_t i)
    {
        it_ += i;
        return *this;
    }
    friend std::ptrdiff_t operator-(
        basic_random_access_iter_dependent lhs,
        basic_random_access_iter_dependent rhs) noexcept
    {
        return lhs.it_ - rhs.it_;
    }

private:
    ValueType * it_;
};

using basic_random_access_iter_dependent_category =
    basic_random_access_iter_dependent<int>::iterator_category;

BOOST_STL_INTERFACES_STATIC_ASSERT_CONCEPT(
    basic_random_access_iter_dependent<int>, std::random_access_iterator)
BOOST_STL_INTERFACES_STATIC_ASSERT_ITERATOR_TRAITS(
    basic_random_access_iter_dependent<int>,
    std::random_access_iterator_tag,
    std::random_access_iterator_tag,
    int,
    int &,
    int *,
    std::ptrdiff_t)
```

This code is well-formed when built with GCC 12 in C++20 mode. The `using basic_random_access_iter_dependent_category` line indicates explicitly that the `iterator_category` is visible. The following macro lines do so implicitly – those are peace-of-mind macros that check that your iterator meets the requirements expected by the STL.

However, the names of typedefs like `iterator_category` is unavailable *inside* the definition of an iterator like `basic_random_access_iter_dependent`. So, If I had used `iterator_category` directly when writing it, like this:

```
template<typename ValueType>
struct basic_random_access_iter_dependent
    : boost::stl_interfaces::iterator_interface<
          std::random_access_iterator_tag,
          ValueType>
{
    static_assert(iterator_category == std::random_access_iterator_tag);

    basic_random_access_iter_dependent() {}
    // etc.
};
```

… the `static_assert` would have been ill-formed.

This is less-than-ideal ergonomically, but it cannot be helped by changing the definition of `iterator_interface`. You can work around this shortcoming by repeating the typedefs in your derived iterator type's definition, or by avoiding the use of those typedefs directly.

3.9.2 The name `iterator_interface`

Also during the 2023-01-17 Library Evolution telecon, someone suggested the alternate name `iterator_facade`. I have received no other recommendations for alternate names. LEWG may want to poll on which of these two names it prefers.

4 Performance

This library's goal is not to provide high performance, so much as it is to provide convenience – to make iterators easy to write.

However, this library is unlikely to have much impact on performance, since `iterator_interface`'s operations are all short inline functions that simply forward to a provided operation. Optimizers are pretty good at optimizing away function calls like that.

As with any performance claim, this should be verified via measurement. Since the provided operations come from outside `iterator_interface`, measuring any one (or N) specializations of `iterator_interface` will not give the full story of all possible performance scenarios. So, I have not tried to characterize the performance cost of using `iterator_interface` here.

Users that require maximum performance should of course measure the performance of their code that uses `iterator_interface`, and, if it is found wanting, reimplement their code without using `iterator_interface`.

5 References

[P2542R2] Hui Xie, S. Levent Yilmaz. 2022-05-11. views::concat.
https://wg21.link/p2542r2